

DOCUMENTO DE ASEGURAMIENTO DE CALIDAD

Administrador Simplificado de Recursos — Proyecto 4

Autor: Fabrizio Espinoza Arce **Curso:** TIIT2007 – Sistemas Operativos, Universidad Invenio **Fecha:** Septiembre de 2026 (revisión 1.1)

1. Propósito y alcance

Este documento describe cómo se aseguró la calidad del Administrador Simplificado de Recursos y qué evidencia respalda cada afirmación. Su función es permitir que un tercero **compruebe** lo que se declara, en lugar de creerlo.

Cubre el código fuente, la suite de pruebas, la evidencia experimental de `results/` y los dos documentos entregables.

La regla que gobierna todo el documento: **se declara lo que la evidencia sostiene, ni más ni menos**. Lo que no se verificó aparece en la sección 10, Limitaciones declaradas, en lugar de omitirse.

2. Identificación del producto de software

Atributo	Valor
Nombre	Administrador Simplificado de Recursos
Versión	1.1.0 (corrige DEF-006 a DEF-009)
Lenguaje	C++17
Dependencias externas	Ninguna
Ejecutable	<code>build/recursos.exe</code> (Windows), <code>build/recursos</code> (Linux)
Suite de pruebas	30 pruebas, la misma en ambas plataformas
Plataformas verificadas	Windows 11 x64 · Ubuntu 26.04 (WSL2)
Equipo de pruebas	AMD Ryzen 5 5600 (6 núcleos, 12 procesadores lógicos), 15,9 GB
Compiladores	MSVC 19.51.36231 · g++ 15.2.0

3. Estrategia de calidad

Cada mecanismo produce evidencia comprobable por máquina, no por revisión visual.

Id	Mecanismo	Evidencia	Estado
M1	Compilación sin advertencias, <code>/W4</code> y <code>-Wall</code> - <code>Wextra</code> - <code>Wpedantic</code>	<code>results/compatibilidad_linux.txt</code>	0 advertencias
M2	Suite de 30 pruebas con arnés propio	<code>results/pruebas_windows.txt</code>	30/30
M3	La misma suite ejecutada en Linux	<code>results/compatibilidad_linux.txt</code>	30/30
M4	Todo lo específico del sistema en un solo directorio	<code>results/pureza_plataforma.txt</code>	Salida vacía
M5	Los tres casos de error del RNF-1, provocados de verdad	<code>results/errores_manejados.txt</code>	3/3
M6	Mediciones con $\text{media} \pm \sigma$ y aviso de fiabilidad	<code>results/csv/</code>	Cumple
M7	La carga del RE-2 se comprueba antes y después de medir	<code>results/prueba_carga.txt</code>	Memoria 85 %, CPU 100 %
M8	Coherencia del documento IEEE por script, incluido el año de cada referencia	<code>scripts/verificar_documento.py</code>	Cumple

Id	Mecanismo	Evidencia	Estado
M9	Reproducibilidad con un comando por plataforma	scripts/*.ps1	Cumple
M10	Lecturas de la herramienta contrastadas con los comandos del sistema (Laboratorio 6)	Sección 9.2	Coinciden

3.1 Qué NO se hace, y por qué

No se usa framework de pruebas. En la máquina de desarrollo no hay gestor de paquetes de C++. El arnés propio ocupa unas 90 líneas y cubre lo que este producto necesita. El coste aceptado es no tener las facilidades de un framework maduro.

No se mide cobertura de código. No hay herramienta disponible sin instalar software adicional. En lugar de dar un porcentaje que no se puede respaldar, la sección 5.2 enumera qué se prueba y qué no.

No se leen las listas de control de acceso de Windows. Requeriría la API de seguridad de Windows y añadiría una superficie considerable para un dato que el enunciado pide como «permisos básicos». Se usa lo que da el estándar de C++ y se declara con precisión qué significa en cada sistema.

4. Matriz de trazabilidad de requisitos

4.1 Requisitos funcionales

Id	Requisito	Módulo	Evidencia	Estado
RF-1	Crear, listar, eliminar y ver metadatos en directorio controlado	plataforma/Archivos	Pruebas 1.1–1.4, 2.1–2.6; results/vistas_menu.txt	Cumple
RF-2	Listar procesos con PID, nombre y consumo	plataforma/Sistema*	Pruebas 5.1–5.6; results/vistas_menu.txt	Cumple
RF-3	Memoria total, usada y disponible	plataforma/Sistema*	Pruebas 4.1–4.3; results/vistas_menu.txt	Cumple
RF-4	Menú de navegación entre las tres funciones	io/Menu	results/vistas_menu.txt; EJECUTAR.bat (menú interactivo); EJECUTAR (Demostraciones).bat	Cumple

4.2 Requisitos no funcionales

Id	Requisito	Solución	Evidencia	Estado
RNF-1	Errores robustos: inexistente, permisos, comando no disponible	Excepciones con mensaje que dice la condición; permission_denied se reconoce y se nombra	Pruebas 3.1 (inexistente), 3.4 (permisos), 5.4 (comando); results/errores_manejados.txt	Cumple
RNF-2	Documentar si es Windows, Linux o ambas	Ambas, las dos ejecutadas	results/compatibilidad_linux.txt	Cumple

4.3 Resultados experimentales obligatorios

Id	Resultado	Evidencia	Estado
RE-1	Tiempos con 10, 100 y 1000 archivos	Tablas 1 y 2 del IEEE; results/csv/rendimiento_*.csv	Cumple
RE-2	Comportamiento bajo carga elevada, comprobada	Tablas 3 y 4 del IEEE; results/prueba_carga.txt; results/csv/carga_*.csv	Cumple
RE-3	Consumo de recursos de la propia herramienta	Tabla 5 del IEEE; results/autoconsumo.txt	Cumple

5. Plan y resultados de pruebas

La suite se ejecuta con `build.bat tests` en Windows y con `make tests` en Linux, y devuelve 0 si todas pasan. Se comprueba el **código de salida**, no el texto del conteo, para no tener que retocar la verificación al añadir pruebas.

Resultado actual: 30 de 30 pruebas superadas en ambas plataformas.

Grupo	Contenido	Pruebas
1	Gestión de archivos (RF-1)	4
2	Frontera del directorio controlado	6
3	Contratos de error (RNF-1)	4
4	Memoria del sistema (RF-3)	3
5	Procesos del sistema (RF-2)	6
6	Utilidades	7
	Total	30

5.1 El desafío metodológico de este proyecto

A diferencia del Proyecto 3, donde la simulación era determinista y se podía comparar contra valores exactos, aquí casi todo lo que se mide depende del estado real de la máquina: cuánta memoria hay libre ahora, cuántos procesos corren en este instante. No se puede escribir `afirmar(procesos == 358)`.

La suite se apoya en tres clases de aserción que sí son estables:

1. **La gestión de archivos**, que es determinista y está bajo control: cada prueba crea su propio directorio temporal y lo borra al terminar.
2. **Los contratos de error**, que se cumplen siempre.
3. **Las invariantes sobre los datos del sistema**. No se afirma cuánta memoria hay, sino que usada + disponible = total y que el porcentaje cae en [0, 100]. Eso es cierto en cualquier máquina y en cualquier momento.

5.2 Lo que se prueba y lo que no

Se prueba: las cuatro operaciones de archivo; la frontera del directorio controlado, incluyendo rutas relativas, absolutas, un directorio hermano de nombre parecido y nombres que designan la propia raíz; los tres casos de error del RNF-1, el de permisos retirando el permiso de verdad; las invariantes de memoria; el análisis de la lista de procesos, incluidos nombres con espacios y el separador de miles; la identificación de la plataforma; el cronómetro y la estadística.

La lista de procesos **se pagina** en el menú (15 por página, con anterior y siguiente) y admite `--top N` y `--todos` en la línea de comandos; las pruebas 6.4 y 6.5 cubren el cálculo de páginas, los bordes y el listado sin recorte.

Cada prueba nueva se comprobó contra el código anterior a la corrección: 2.6 y 3.4 fallan con la versión 1.0.0 y pasan con la 1.1.0. Una prueba que pasa antes y después de corregir un defecto no demuestra nada.

No se prueba automáticamente: el formato exacto de la salida en consola, porque su única salida observable es un flujo y probarla exigiría capturarlo; y la apariencia de los `.docx` y `.pdf`, que se revisó manualmente página por página.

5.3 La costura para probar «comando no disponible»

El enunciado exige manejar el caso de comando no disponible. No se puede provocar desinstalando `tasklist` del sistema, así que la capa de plataforma expone `listarProcesosCon(comando)`, una variante que acepta otro comando. La prueba 5.4 la llama con uno inexistente y comprueba que se lanza una excepción cuyo mensaje nombra el comando y explica la causa probable.

Esa costura existe **exclusivamente** por esa razón, y no por generalidad: sin ella el caso se podría afirmar pero no probar, y este proyecto no publica afirmaciones sin respaldo. La prueba 5.6 la reutiliza para alimentar el análisis con una salida controlada: un proceso llamado «Web Content» y una memoria con separador de miles.

5.4 Cómo se provoca «permisos insuficientes»

La prueba 3.4 retira el permiso de escritura sobre un directorio con la herramienta nativa de cada sistema —`icacls /deny *S-1-1-0:(W,AD)` en Windows, `bits r-x` en Linux— e intenta crear un archivo dentro. Se usa el SID del grupo «Todos» y no su nombre, que cambia con el idioma de Windows. El permiso se restaura en un destructor, pase lo que pase. Si el usuario puede escribir igualmente —`root` en Linux ignora los bits—, la prueba se anuncia como omitida en vez de fingir el caso; en la verificación de WSL, que corre como usuario normal, se ejecutó.

6. Gestión de defectos

DEF-001 — La prueba de carga daba resultados invertidos

Campo	Detalle
Detectado	Al revisar los factores del RE-2 y ver valores menores que 1
Severidad	Alta. El resultado era plausible a simple vista y falso
Causa	La medición sin carga corría primero, en frío, y la medición con carga después, con la caché del sistema de archivos ya caliente. El sesgo favorecía a la segunda
Impacto	El documento habría afirmado que la herramienta es más rápida con el sistema cargado
Corrección	Pasada de calentamiento descartada antes de medir, para que ambas condiciones partan iguales
Estado	Cerrado

DEF-002 — La carga de memoria no se aplicaba

Campo	Detalle
Detectado	La comprobación de carga reportó 8 MB ocupados en vez de 1,5 GB
Severidad	Media. Habría convertido el RE-2 en una prueba solo de CPU sin decirlo
Causa	Reservar un arreglo no confirma las páginas de memoria: el sistema no las asigna hasta que se tocan
Corrección	El generador de carga ahora escribe un byte por página de 4 KB, lo que fuerza la confirmación
Prevención	El script mide y registra la memoria ocupada antes y durante, de modo que una carga que no se aplique quede a la vista
Estado	Reabierto como DEF-007: la corrección no bastó, y la evidencia publicada lo mostraba

DEF-003 — El menú rechazaba la primera opción al automatizarse

Campo	Detalle
Detectado	Al conducir el menú por tubería para generar evidencia
Severidad	Media. Impedía automatizar las demostraciones
Causa	PowerShell antepone un BOM de UTF-8 al escribir a una tubería. La opción «1» llegaba como tres bytes invisibles más el 1
Síntoma engañoso	El programa decía que «1» no era una opción válida, mostrando lo que parecía un 1. Solo se ve mirando los bytes
Corrección	La normalización de entrada descarta el BOM si viene al frente
Estado	Cerrado

DEF-004 — Error de sintaxis reportado 55 líneas después de su causa

Campo	Detalle
Detectado	Al ejecutar verificar_linux.ps1 por primera vez
Severidad	Baja. Impedía ejecutar el script, de forma ruidosa
Causa	Se escribió cut -d'\ "' dentro de una cadena de PowerShell. En PowerShell la barra invertida no escapa una comilla —el escape es el acento grave— así que la comilla cerró la cadena
Síntoma engañoso	El parser reportó el error en la línea 149, cuando la causa estaba en la 94
Corrección	Se eliminó toda comilla doble del interior de esa cadena
Estado	Cerrado

DEF-005 — La lista de distribuciones de WSL se leía como una sola letra

Campo	Detalle
Detectado	El script reportó la distribución «U» en vez de «Ubuntu»
Causa	ws1.exe emite UTF-16LE; al capturarlo desde PowerShell cada letra queda separada por un byte nulo
Corrección	Se define WSL_UTF8=1 antes de invocarlo
Estado	Cerrado

DEF-006 — «Eliminar .» borraba el propio directorio de trabajo

Campo	Detalle
Detectado	Revisión posterior a la versión 1.0.0, probando nombres límite en el menú
Severidad	Alta. Operación destructiva sobre la frontera que la herramienta promete proteger
Causa	La comprobación de frontera exigía que la ruta empezara por la raíz, y la raíz empieza por sí misma. Con el directorio vacío, fs::remove lo borraba y el menú respondía «Eliminado '.'»
Corrección	resolverSeguro exige al menos un componente propio después de la raíz; ., ./ y sub/. se rechazan
Prueba	2.6, que falla con la versión 1.0.0
Estado	Cerrado

DEF-007 — La carga del RE-2 no estaba aplicada al medir

Campo	Detalle
Detectado	Revisión de la evidencia: prueba_carga.txt registraba 91 MB ocupados, cuando el texto decía 1,5 GB. Al corregirlo apareció un segundo problema: la CPU estaba al 9 % al empezar a medir
Severidad	Alta. El RE-2 se publicó como medido «bajo carga elevada» sin que la carga existiera, y DEF-002 constaba como cerrado
Causa	Las cargas se lanzaban a la vez y se esperaba un tiempo fijo. El trabajo de memoria tocaba cada página desde un bucle interpretado, frenado además por los doce bucles de CPU; y los doce trabajos de CPU tardan más de 10 s en arrancar
Corrección	La memoria se reserva primero, las páginas se tocan desde C# compilado, y el trabajo avisa al terminar. La cantidad se calcula para llevar la ocupación al 85 %. Los trabajos de CPU avisan al entrar en su bucle, y no se mide hasta que avisaron todos y el uso de CPU lo confirma
Prevención	Se registran memoria y CPU al empezar y al terminar de medir, y la evidencia lleva un aviso escrito si la carga no alcanza el objetivo
Resultado	Memoria al 84,9 %, CPU al 100 % al empezar y al terminar, 13 de 13 trabajos vivos
Estado	Cerrado, con evidencia

DEF-008 — «Permisos insuficientes» no se probaba y su mensaje confundía

Campo	Detalle
Detectado	Revisión de la trazabilidad: el comentario del grupo 3 decía que cada caso del RNF-1 tenía prueba, y el de permisos no la tenía. En errores_manejados.txt se sustituía por «salir del directorio» y «opción inválida»
Severidad	Media. Un requisito declarado como cumplido sin evidencia
Síntoma	Con la escritura denegada, el mensaje era «no se pudo interpretar el nombre 'nuevo.txt': Access is denied», que habla del nombre y no de permisos
Corrección	Los errores del sistema de archivos se comparan con std::errc::permission_denied y el mensaje dice «permisos insuficientes». Prueba 3.4 y caso 2 de errores_manejados.txt, ambos con el permiso retirado de verdad
Estado	Cerrado

DEF-009 — Defectos menores encontrados en la misma revisión

Defecto	Corrección
En Linux, un proceso con espacios en el nombre se descartaba en silencio	El nombre es lo que queda entre el PID y la última columna; prueba 5.6
--cantidad 5abc se aceptaba como 5	Se exige que todo el valor sea numérico
Dockerfile.linux llamaba a un script inexistente	Se añadió scripts/verificar_linux_interno.sh
verificar_linux.ps1 daba por buena la suite solo por encontrar la línea del conteo	Ahora falla si aparece algún [FALLA]
El documento IEEE citaba las referencias sin año y aludía a una tabla equivocada	Corregido; verificar_documento.py comprueba ahora el año
Comentarios heredados de otros proyectos en Estadistica.h y Makefile	Actualizados

7. Métricas objetivas del producto

7.1 Métricas de construcción

Métrica	Valor
Ficheros de código C++	17
Líneas de C++ propio	2 748
— src/core/ (lógica pura)	154
— src/plataforma/	875
— de las cuales, específicas de un sistema	382
— src/io/	696
— src/main.cpp	283
— suite de pruebas	740
Líneas de scripts	1 002
Advertencias, MSVC /W4	0
Advertencias, g++ -Wall -Wextra -Wpedantic	0

Las líneas se cuentan con `wc -l`, no con `Measure-Object -Line` de PowerShell, que subcuenta: en el Proyecto 2 se publicaron 711 líneas cuando eran 802.

El dato más significativo de esta tabla son las 382 líneas específicas de plataforma sobre 2 748 totales: el 14 %. Ese porcentaje es la medida concreta de lo que costaría portar la herramienta a un tercer sistema operativo.

7.2 Métricas de comportamiento

Métrica	Valor
Pruebas automatizadas	30
Pruebas superadas en Windows	30
Pruebas superadas en Linux	30
Plataformas verificadas	2
Defectos detectados y cerrados	9
Defectos abiertos	0
Memoria residente de la herramienta	4,0 MB (0,024 %)
Ralentización media bajo carga	2,06 (entre 1,62 y 3,29)

7.3 Fiabilidad de las mediciones de tiempo

Se usa `std::chrono::steady_clock`, garantizado como monótono, y no `system_clock`, que puede retroceder si el sistema ajusta la hora.

Cada medición se repite cinco veces y se reporta media y desviación muestral. **El reporte advierte automáticamente** cuando la desviación supera la mitad de la media, señalando que a esa escala la medición está dominada por ruido. Ese aviso se dispara con 10 y 100 archivos en la operación de listar, cuyo tiempo es de décimas de milisegundo, y es correcto que lo haga. **Esas filas no se usan para sacar conclusiones:** la comparación entre listar y consultar metadatos del documento IEEE se apoya solo en la fila de 1000 archivos.

Antes de medir se ejecuta una pasada de calentamiento que se descarta. Sin ella el resultado de la prueba de carga salía invertido (DEF-001).

8. Reproducibilidad

Toda la evidencia se regenera con dos comandos:

```
powershell -ExecutionPolicy Bypass -File scripts\correr_experimentos.ps1
powershell -ExecutionPolicy Bypass -File scripts\verificar_linux.ps1
```

Paso	Qué hace	Resultado esperado
0	Localiza Visual Studio, normaliza los .bat y compila	build\recursos.exe
1	Ejecuta la suite	30/30 pruebas superadas
2	Busca windows.h fuera de plataforma/	Salida vacía
3	Captura las tres vistas del menú	RF-1, RF-2 y RF-3
4	RE-1 con 10, 100 y 1000 archivos	Tablas + CSV
5	RE-3, consumo propio	4 MB aproximadamente
6	RE-2: aplica y comprueba la carga, y repite las medidas	Memoria cerca del 85 %, CPU cerca del 100 %; factores cercanos a 2
7	Provoca los tres casos de error, el de permisos con <code>icac1s</code>	Mensajes claros, sin caída
8	Limpia el área temporal	Sin residuos

El script de experimentos **aborta si la suite no pasa**: no se generan resultados sobre un binario que falla sus propias pruebas.

8.1 Qué es y qué no es reproducible

Reproducible: el comportamiento funcional completo, los mensajes de error, y el hecho de que el mismo código compile y pase las pruebas en ambos sistemas.

No reproducible exactamente: los tiempos y las cifras de memoria y procesos, que dependen de la máquina y del instante. Por eso se reportan con dispersión y por eso las conclusiones se apoyan en relaciones —el coste por archivo es estable, listar por lote es unas cien veces más barato que consultar uno a uno, la ralentización ronda el doble— y no en valores absolutos.

9. Evaluación ISO/IEC 25010

Rendimiento — nivel 4. El criterio es «mide y reporta tiempos y recursos con datos concretos y los compara entre escenarios». Se miden cuatro operaciones en tres escalas, con y sin carga del sistema —comprobada, no supuesta—, y se mide el consumo propio. El reporte advierte cuando una medición no es fiable, y esas filas no se usan para concluir.

Fiabilidad — nivel 4. El criterio es «maneja errores explícitamente y produce resultados consistentes en ejecuciones repetidas». 30 pruebas que pasan en dos plataformas, los tres casos de error del enunciado provocados de verdad, y nueve defectos documentados y cerrados, cada uno de los de código con su prueba de regresión.

Usabilidad — nivel 4. La pregunta guía es «¿la interfaz de consola es comprensible para un usuario nuevo?». Menú con submenú, ayuda con ejemplos, confirmación antes de eliminar, mensajes que dicen qué se intentaba y por qué falló, y una opción inválida que no interrumpe el programa.

Compatibilidad — nivel 4. El criterio es «compila y corre en múltiples entornos o declara limitaciones con justificación». Aquí se cumple la primera rama sin recurrir a la segunda: el mismo código fuente compila sin advertencias y pasa las 30 pruebas en Windows con MSVC y en Linux con g++.

Mantenibilidad — nivel 4. El criterio es «código modular, comentado, con nombres descriptivos». El 86 % del código es independiente del sistema operativo, y esa separación se verifica automáticamente.

Característica	Nivel	Evidencia principal
Rendimiento	4	results/csv/
Fiabilidad	4	results/pruebas_windows.txt
Usabilidad	4	results/vistas_menu.txt
Compatibilidad	4	results/compatibilidad_linux.txt
Mantenibilidad	4	results/pureza_plataforma.txt

9.1 Evidencia de compatibilidad

Aspecto	Windows	Linux
Sistema	Windows 11 Pro 10.0.26100	Ubuntu 26.04 LTS (WSL2)
Compilador	MSVC 19.51.36231	g++ 15.2.0
Banderas	/std:c++17 /W4 /O2	-std=c++17 -O2 -Wall -Wextra -Wpedantic
Advertencias	0	0
Suite	30/30	30/30

Alcance declarado con precisión. WSL2 usa un núcleo Linux real, así que /proc/meminfo y ps son los de Linux y el código ejercitado es el mismo que correría en una máquina Linux nativa. Lo que WSL no reproduce es el hardware ni la carga de un servidor real: por eso las **cifras** de las dos plataformas no son comparables entre sí en sentido estricto. Lo que sí queda demostrado es que el mismo código fuente funciona en ambas.

9.2 Validación con el Laboratorio 6

En el Laboratorio 6 (Semana 10) se midieron las mismas magnitudes con los comandos nativos de cada sistema y se contrastaron con la herramienta en el mismo instante. La evidencia está en la carpeta del laboratorio (Laboratorios/semana 10/evidencia/, archivo windows/contraste_p4.txt y celda S14c del cuaderno).

Magnitud	Proyecto 4	Sistema	Resultado
Linux, memoria usada	635,2 MB	free: 636,0 MB	Coincide
Linux, RSS del proceso 384	107,5 MB	ps: 110 044 kB = 107,5 MB	Idéntico
Windows, memoria usada	61,8 %	Win32_OperatingSystem: 61,8 %	Idéntico
Windows, memoria disponible	6,1 GB	Get-Counter: 6239 MB	Coincide
Windows, msedgewebview2	615,8 MB	WS 615,8 MB · WS privado 546,8 MB	Igual al WS
Windows, explorer	397,4 MB	WS 397,4 MB · WS privado 106,4 MB	Igual al WS

La validación confirma que la herramienta **mide lo que el sistema reporta**, y precisa una cosa que no estaba documentada: la memoria por proceso es el *working set* completo (en Linux, el RSS), con las páginas compartidas incluidas. Se añade como limitación 8.

10. Limitaciones declaradas

1. **La verificación de Linux se hizo sobre WSL2, no sobre una máquina Linux nativa.** El núcleo es real y el código ejercitado es el mismo, pero el hardware y el entorno no son los de un servidor.
2. **Los permisos que se muestran en Windows no son los reales.** La biblioteca estándar de C++ no traduce las listas de control de acceso y devuelve el permiso más amplio. En Linux sí son los bits reales del inodo. La herramienta lo advierte en su propia salida.
3. **En Windows, la biblioteca estándar elimina también los archivos de solo lectura.** El atributo de solo lectura no es una ACL, y `std::filesystem::remove` de MSVC lo ignora. Una ACL que deniega la escritura sí se respeta y se informa como «permisos insuficientes».
4. **El monitoreo es una fotografía instantánea,** no un seguimiento continuo.
5. **No se mide cobertura de código:** no hay herramienta disponible sin instalar software adicional.
6. **Los tiempos se midieron en una sola máquina.** Las comparaciones relativas —coste por archivo, factor de degradación— son válidas; los valores absolutos no son extrapolables.
7. **La carga del RE-2 es sintética.** Doce procesos de CPU y un proceso que reserva memoria no reproducen el perfil de un servidor en producción, pero sí producen contención real, medible y comprobada.
8. **La memoria por proceso incluye las páginas compartidas.** Es el *working set* completo de `tasklist` y el RSS de `ps` (sección 9.2). Por eso la «memoria sumada de todos» cuenta varias veces las bibliotecas compartidas y no equivale a la memoria usada del sistema, que es la que muestra la opción 3 del menú. La herramienta lo advierte en esa misma línea de su salida.

11. Conclusión

El producto cumple los cuatro requisitos funcionales, los dos no funcionales y los tres resultados experimentales obligatorios, con evidencia comprobable para cada uno (sección 4).

La diferencia respecto a los tres proyectos anteriores está en la Compatibilidad. En el Proyecto 2 y en el Proyecto 3 la rama de Linux se entregó escrita pero nunca ejecutada, y hubo que declararlo como limitación. Aquí la misma suite corre en los dos sistemas, y el 86 % del código es independiente del sistema operativo.

Los nueve defectos de la sección 6 son parte del resultado, no una mancha. Tres de ellos —DEF-001, DEF-002 y DEF-007— produjeron o habrían producido un resultado experimental plausible y falso, que es la peor clase de error posible en un documento que presume de datos medidos. DEF-007 enseña además que un defecto no está cerrado

porque se haya aplicado una corrección, sino cuando la evidencia lo demuestra: DEF-002 se dio por cerrado mientras la propia evidencia decía lo contrario.

12. Referencias

- [1] Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). *Operating System Concepts* (10.^a ed.). John Wiley & Sons.
- [2] ISO/IEC. (2011). *ISO/IEC 25010:2011 — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*.
- [3] Universidad Invenio. (2026). *Rúbricas de Evaluación TIIT2007*, Rúbrica 2 — ISO/IEC 25010.
- [4] Universidad Invenio. (2026). *Especificaciones de Proyectos TIIT2007*, Proyecto 4 — Administrador Simplificado de Recursos.
- [5] Universidad Invenio. (2026). *Dataset Experimental TIIT2007*, Dataset 4.

Declaración de verificación final

Todo dato numérico de este documento y del documento IEEE procede de un fichero de results/ generado por los scripts de reproducción. Ninguno se estimó, se recordó ni se redondeó para que encajara.

Las afirmaciones se comprobaron en la fecha de emisión ejecutando:

```
powershell -ExecutionPolicy Bypass -File scripts\correr_experimentos.ps1
powershell -ExecutionPolicy Bypass -File scripts\verificar_linux.ps1
python scripts\verificar_documento.py
```

Con resultado: **30/30 pruebas superadas en Windows, 30/30 en Linux, cero advertencias en ambos compiladores, windows.h confinado en src/plataforma/, carga del RE-2 comprobada (memoria 84,9 %, CPU 100 %) y documento IEEE coherente.**